Prácticas

Node JS



Sistemas Distribuidos

Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA Y COMPUTACIÓN

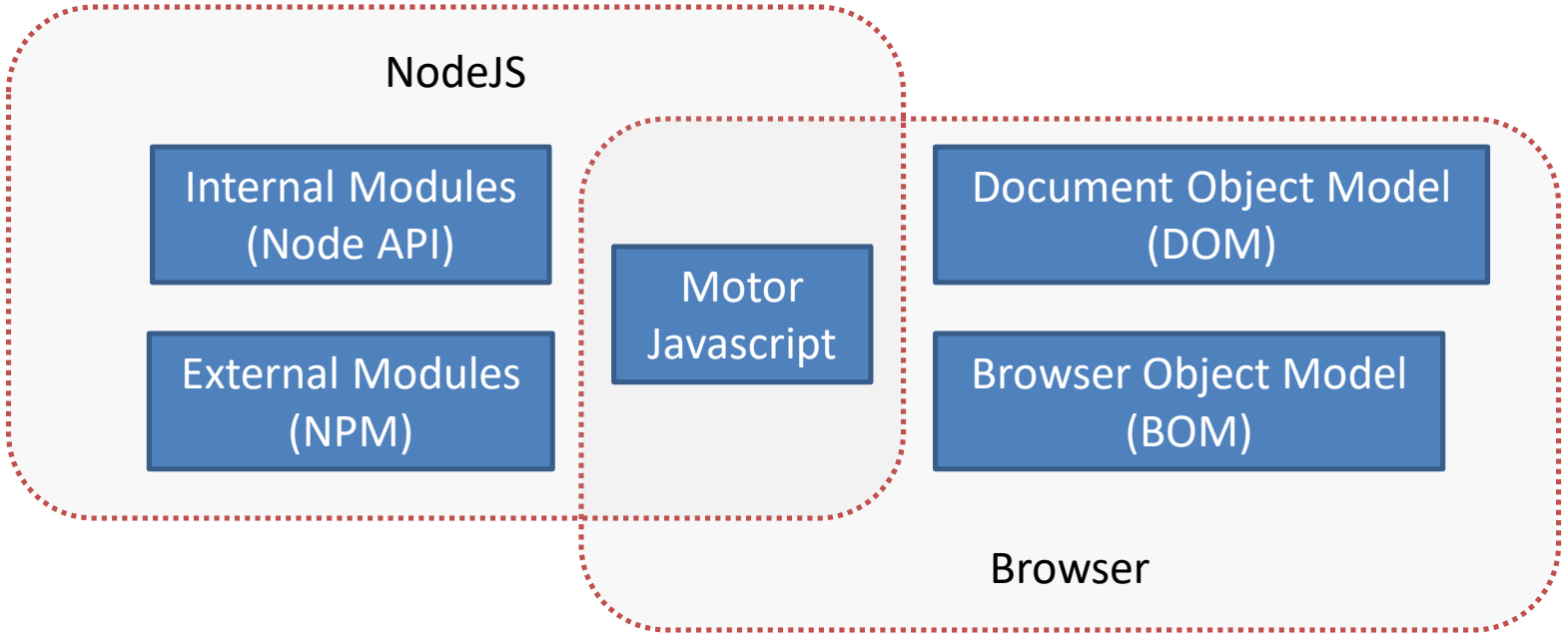
IB



Introducción

- **NodeJS**
  - Entorno en tiempo de ejecución (interpretación)
  - Basado en Javascript (ECMAScript)
  - Permite la creación de aplicaciones Javascript en el servidor
  - Utiliza V8, el motor JS de Chrome
  - Programación asíncrona orientada a eventos
  - Sistema modular
  - Único hilo de ejecución (evita problemas de concurrencia)

Arquitectura





Creación de aplicaciones

- Instalar NodeJS
- Crear una carpeta de trabajo
- Crear archivo `.js`
- Lanzar con el comando `node`

```
node miapp.js
```

- Se ejecutan las instrucciones fuera de cualquier definición de función (no hay `main`)



API

API	Descripción
Buffer	Tratamiento de datos en crudo (binarios)
Console	Acceso a la consola (console.log)
Events	Creación / invocación de eventos
FileSystem	Acceso a archivos
HTTP	Servidor / cliente HTTP
Modules	Gestión de módulos
Net	Trabajo con red por flujos (sockets TCP)
OS	Operaciones sobre el sistema operativo
UDP/Datagram	Trabajo en red por paquetes (sockets UDP)
…	

`https://nodejs.org/api/`



Módulos

- Node aporta un sistema modular para organizar el código
- Un modulo es un archivo .js que intruduce el API exportado como miembros del objeto `module.exports`

```
// circle.js
module.exports.area = function (r) { return Math.PI * r * r; };
module.exports.circumference = function (r) { 2 * PI * r; };
```

- Se importa en otro módulo mediante `require`

```
const circle = require('./circle.js');
var a = circle.area(4);
```



Módulos externos

- Junto con Node se instala un gestor de paquetes: **npm** (www.npmjs.com)
- Permite descargar e instalar módulos de terceros
- Desde la carpeta del proyecto:

```
npm install <package_name>
```

- Esto crea un a carpeta `node_modules` con los módulos instalados
- Utilizar la opción `-g` para instalar módulos globales (accesibles a todas las aplicaciones)

Prácticas

Node JS



Sistemas Distribuidos

Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA Y COMPUTACIÓN

IB

Prácticas

Express



Sistemas Distribuidos

Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA Y COMPUTACIÓN

IB



Introducción

- **Express**
  - Infraestructura para la creación de aplicaciones web
  - Módulo de NodeJS
  - API que facilita la creación de sitios y servicios web
  - Alto rendimiento



Instalación

- Se instala como cualquier módulo de Node

- Dentro de la carpeta de la aplicación:

```
npm install express
```



Hola mundo

```
var express = require('express'); // cargar el módulo
var app = express(); // crear la app express

// Contestar a la petición GET sobre /
app.get('/', function (req, res) {
  res.send('Hola mundo');
});

// Iniciar el servidor web en el puerto 3000
app.listen(3000, function () {
  console.log('Servidor escuchando 4en puerto 3000');
});
```

```
node holamundo.js
```



Direccionamiento básico

- Utiliza los métodos de HTTP para enlazar funciones manejadoras
- Las funciones manejadoras tienen siempre los argumentos petición (`req`) y respuesta (`res`)

```
app.METODO(RUTA, FUNCION_MANEJADORA);
```

```
app.get("/", function (req, res) { … });
app.post("/user", function (req, res) { … });
app.put("/user/diego", function (req, res) { … });
```



Archivos estáticos

- Express puede actuar como un servidor web de recursos estáticos (como Apache o IIS)
- Para ello utiliza el método `express.static`

```
app.use(RUTA_URL, express.static(RUTA_CARPETA));

app.use('/web', express.static('/publico'));
```



Direccionamiento

- Para la ruta se puede establecer un literal de cadena, comodines o una expresión regular

```
// Cadena: /about
app.get('/about', function (req, res) { … });

// Comodines: /abCUALQUIERCOSAcd
app.get('/ab*cd', function(req, res) { … });

// Expresion regular: /aa872
app.get(/^aa[0-9]*$/, function(req, res) { … });
```



Encadenar manejadores

- Se pueden invocar varios manejadores para una misma solicitud
- Utilizar el método next (tercer argumento)

```
app.get('/about', function (req, res, next) {
  console.log("Primero");
  next(); // no termina, continua el procesado
});

app.get('/about', function (req, res) {
  res.send("ABOUT");
});
```



Métodos de respuesta

`res.download()`	Solicita un archivo para descargarlo.
`res.end()`	Finaliza el proceso de respuesta.
`res.json()`	Envía una respuesta JSON.
`res.jsonp()`	Envía una respuesta JSON con soporte JSONP.
`res.redirect()`	Redirecciona una solicitud.
`res.render()`	Representa una plantilla de vista.
`res.send()`	Envía una respuesta de varios tipos.
`res.sendFile`	Envía un archivo como una secuencia de octetos.
`res.sendStatus()`	Establece el código de estado de la respuesta y envía su representación de serie como el cuerpo de respuesta.



Manejadores de rutas

- Los manejadores de rutas concentran los accesos a una misma ruta

```
app.route('/book')
  .get(function(req, res) {
    res.send('Get a random book');
  })
  .post(function(req, res) {
    res.send('Add a book');
  })
  .put(function(req, res) {
    res.send('Update the book');
  });
```



Funciones middelware

- Funciones intermedias que realizan un procesado
  - Ejecutar cualquier código
  - Realizar cambios en los objetos `req` y `res`
  - Finalizar el ciclo de solicitud/respuestas
  - Invocar el siguiente middleware en la pila (`next`)
- Se integran en el flujo de procesado mediante **`use`**

```
var express = require('express');
var app = express();

var requestTime = function (req, res, next) {
  req.requestTime = Date.now();
  next();
};

app.use(requestTime);

app.get('/test', function (req, res) {
  res.send("Solicitud de test en: " + req.requestTime);
});

app.listen(3000);
```



JSON

- El método `express.json` permite analizar las solicitudes de peticiones en formato JSON
- La información es almacenada en `req.body`

```
app.use(express.json());
```



Resumen API

Request

Propiedades
req.app
req.baseUrl
req.body
req.cookies
req.fresh
req.hostname
req.ip
req.ips
req.method
req.originalUrl
req.params
req.path
req.protocol
req.query
req.route
req.secure
req.signedCookies
req.stale
req.subdomains
req.xhr
Métodos
req.accepts()
req.acceptsCharsets()
req.acceptsEncodings()
req.acceptsLanguages()
req.get()
req.is()
req.param()
req.range()

Response

Propiedades
res.app
res.headersSent
res.locals
Métodos
res.append()
res.attachment()
res.cookie()
res.clearCookie()
res.download()
res.end()
res.format()
res.get()
res.json()
res.jsonp()
res.links()
res.location()
res.redirect()
res.render()
res.send()
res.sendFile()
res.sendStatus()
res.set()
res.status()
res.type()
res.vary()

Prácticas

Express



Sistemas Distribuidos

Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA Y COMPUTACIÓN

IB

Prácticas

REST



Sistemas Distribuidos

Universitat d'Alacant
Universidad de Alicante

DEPARTAMENTO DE TECNOLOGÍA INFORMÁTICA Y COMPUTACIÓN

GIB



Introducción

- **RE**presentational **S**tate **T**ransfer (**REST**)
  - Etilo arquitectónico para crear sistemas distribuidos basados en servicios web interoperables (*RESTFul services*)
  - No es un estándar (sólo un estilo arquitectónico)
  - Propuesto en 2000 por **Roy Fielding** (tesis doctoral)
  - Utilizado por servicios web que se basan y utilizan exclusivamente el protocolo HTTP
  - Orientación a los recursos (datos) y no a las operaciones
  - Alto éxito y crecimiento en la actualidad



Definición

- REST utiliza el protocolo HTTP como base de su arquitectura
  - Arquitectura sin estados
  - Trabaja con recursos
  - Cada recurso está identificado por una URI
  - Las operaciones son mapeadas directamente a los métodos HTTP
    - GET, POST, PUT, DELETE, PATCH…
  - La información es codificada en el cuerpo HTTP (XML, JSON, HTML…)



Identificación

- Utiliza la identificación nativa de HTTP mediante URIs
- Tipos
  - Recursos simples (**Elemento**)
    - Referencia un recurso que puede ser leído, actualizado o eliminado
    - Ejemplos
      - `/user/luis`
      - `/product/89583`
  - Listas/Arrays de recursos (**Colecciones**)
    - Referencia un conjunto de recursos sobre los que adicionalmente se puede insertar nuevos recursos
    - Ejemplos
      - `/user` -> listado de usuarios
      - `/product` -> listado de productos
      - `/person` -> listado de personas
      - `/person/andrew/children` -> listado de personas que son hijos de Andrew
- La estructura y semántica del direccionamiento es definida en función de la aplicación



Operaciones

- Suele ser habitual asociar operaciones básicas de **CRUD** sobre los recursos
  - ***C**reate, **R**ead, **U**pdate & **D**elete*
- Utiliza los métodos de HTTP para realizar estas operaciones

Método HTTP	Operación	Ejemplo
GET	Obtener (leer) recurso	GET /user/luis
POST	Crear / modificar recurso	POST /user
PUT	Reemplaza (sobreescribir) recurso	PUT /product/45
PATCH	Actualiza (parcial) el precurso	PATCH /product/45
DELETE	Eliminar recurso	DELETE /inform/anual2018



Operaciones

Operación	Elemento	Colección
GET	Obtiene una representación del elemento	Obtiene un listado de elementos: URIs y/o datos
PUT	Reemplaza el elemento. Si no existe lo crea.	Reemplaza la colección
PATCH	Actualiza el elemento	Actualiza la colección
POST	-	Crea una nueva entrada
DELETE	Borra el elemento	Borra la colección



Operaciones

- GET es una operación **segura** o ***nullipotent***
  - Su llamada no tiene efectos secundarios
  - Acceder / leer a un recurso tiene el mismo efecto sobre él que no haberlo hecho nunca
- PUT y DELETE son operaciones **idempotentes**
  - El estado del sistema no cambia en función del número de veces que se invoque la operación



Información / Codificación

- La información solicitada en las lecturas (GET), o las enviadas en las actualizaciones/creaciones son enviadas en el cuerpo del mensaje HTTP

- Al igual que en HTTP el formato es configurable
  - Definido en la cabecera HTTP `Content-Type`
  - El formato más utilizado en la actualidad es **JSON**



Características

- Hereda las características de HTTP
  - **Alto rendimiento**
    - Operaciones atómicas con baja latencia
    - No incluye capas adicionales
    - Fácil análisis e interpretación
  - **Escalabilidad**
  - **Confiabilidad** y **tolerancia a fallos**
  - **Simplicidad** e **uniformidad** en la interfaz
  - **Flexibilidad**
  - **Portabilidad**



Restricciones de la arquitectura

- Arquitectura Cliente-servidor
  - Separación de las labores de
    - Presentación -> cliente
    - Modelo y datos -> servidor
- Arquitectura sin estado
  - El servidor no almacena contexto sobre el cliente
  - Peticiones independientes y autocontenidas
- Cache
  - Clientes e intermediaros pueden cachear la información
  - En las respuestas hay que indicar si la información puede ser o no cacheada



REST vs. SOAP

REST	SOAP
Usa HTTP a nivel de aplicación	Usa HTTP (u otro protocolo) como protocolo de comunicación
Orientado a los recursos (pocas operaciones con muchos recursos)	Orientado a las operaciones (muchas operaciones con pocos recursos)
Simple / Ligero	Complejo / Pesado
HTTPS	WS SECURITY